

Agent Hooks Core: portable semantics for coding-agent hooks

A proposal from the Weft project for the Agentic AI Foundation (AAIF) · October 2026 · MIT

Problem. Every major coding-agent runtime now has hooks: Claude Code, OpenAI Codex, Gemini CLI, Cursor and GitHub Copilot all document ways to observe a tool call, add context for the model, deny a call before it runs, and refuse to let the agent stop. None of them are interoperable. Event names, payloads, output fields, timeouts and failure behaviour all differ, so a policy, audit or coordination plugin has to be rewritten for each host. A hook that "checks edits" can enforce in one runtime and only advise in another, and the plugin has no standard way to say which.

Proposal. A small, transport-agnostic **Agent Hooks Core** ([spec](#)):

1. **Capability declaration:** L0 observe, L1 inject, L2 block, L3 gate completion, plus commit gate and failure policy. A hook must not declare a level it does not deliver.
2. **Normalized lifecycle events:** `session.start`, `prompt.submit`, `tool.pre`, `tool.post`, `agent.stop`, `session.end`, with tool kinds (edit, write, shell, ...) and call ids.
3. **Decision envelope:** `allow`, `advise` or `deny`, and an explanation that must reach the model.
4. **Diagnostic model:** severity, code, resource, cause and suggestion.
5. **Timeout and failure policy, security and privacy:** fail-open or fail-closed is declared, invalid output is never a silent policy, and injected text is treated as a prompt.
6. **Host mappings and a conformance suite:** a black-box runner that tests any hook command at its native wire and reports the verified level.

Why now. Agent Plugins 1.0 (August 2026) left hooks out on purpose: they "do not yet have consistent semantics or security models across clients". Its rule for adding a component type is a concrete cross-client need plus evidence that implementers can support the same semantics. The runtimes already agree on what hooks mean, but each spells it differently. Standardizing the meaning now costs little. Waiting means plugin authors keep writing one adapter per host.

Evidence. These come from a working system, not a paper design:

- **Weft**, a coordination layer for coding agents, is built on these semantics and coordinated its own build. Adapters for **Claude Code**, **Codex**, **OpenCode** and **Hermes** run live: Claude Code and OpenCode are verified at L0–L3, Codex at L0–L2. Translators for Cursor and Gemini CLI are tested against the documented formats only.
- **Conformance:** all five adapters verify at **L3** at the wire, about 100 ms per hook (p50). The suite also caught a real over-declaration: in advisory mode the adapters delivered L1 but declared L3. They now declare L1 there ([reports](#)).
- **Built under itself:** Weft's own log recorded **627 events in 11.3 hours**. That was **460 edits** from 3 agent profiles across 25 tasks, with 73 checkpoints, 18 landings and 59 warnings that told one agent to wait for another.
- **Scale:** in the full end-to-end run, **18 live agents** (6 each of Claude Code, Codex and OpenCode) worked 6 tasks against one coordinator. Every coordination beat passed in all 4 valid runs: collision, live denial and reroute, structural merge, negotiation and landing.

Scope. *In:* the six items above. *Out:* plugin packaging (Agent Plugins), tool access (MCP), agent messaging (A2A), permission UX, sandboxing, and anything a hook decides. Edit-time coordination between agents (WCP) is one optional extension on top. The core does not require it, a server, or any vendor's cloud.

Ask.

1. A discussion slot in an AAIF working group, probably *Workflows & Process Integration with Security & Privacy* for §9, to decide whether hook semantics should be an AAIF work item.
2. Runtime maintainers check the host-mapping table (spec §10) for their product and run the conformance suite against their own hooks.
3. Coordination with the Agent Plugins maintainers, so that a future portable hooks component can point at one set of semantics instead of five.

We are not asking AAIF to adopt Weft, its server or its cloud stack.

Spec: [docs/protocol/hooks-core-v0.md](#) · Conformance: [packages/conformance](#) · 3-minute video: <https://weft-media.elier.ai/weft-demo-short.mp4> · Repository: <https://github.com/celador/weft> · Discussions: <https://github.com/celador/weft/discussions>